Эффективная реализация лямбда-функций в объектно-ориентированных языках программирования

Болохононов Артем
Научный руководитель: Трепаков Иван Сергеевич

Анонимные функции

Определение

```
def filterEven(x: Int): Bool = {
  x % 2 == 0
}
```

Анонимные функции

Определение

```
def filterEven(x: Int): Bool = {
  x % 2 == 0
}

// Анонимные функции (или лямбда-функции) --- концепция, пришедшая из языков функциональной парадигмы
// Она позволяет по ходу программы создавать функции, которые не имеют уникального индентификатора
val filerEvenLambda: Int => Bool = { x: Int => x % 2 }  // == filterEven
```

Анонимные функции

Определение

```
// Анонимные функции (или лямбда-функции) --- концепция, пришедшая из языков функциональной парадигмы
// Она позволяет по ходу программы создавать функции, которые не имеют уникального индентификатора
val filerEvenLambda: Int => Bool = { x: Int => x % 2 }

// Также лямбда-функции обладают возможностью замыкать переменные
// Это позволяет сохранить используемый контекст для дальнейшего исполнения
val capture: Int = 5
val functionalCaputre: => Int = { capture }

...

functionalCapture() // == 5
```

Анонимные функции

Определение

```
// Анонимные функции (или лямбда-функции) --- концепция, пришедшая из языков функциональной парадигмы
// Она позволяет по ходу программы создавать функции, которые не имеют уникального индентификатора
val filerEvenLambda: Int => Bool = { x: Int => x % 2 }  // == filterEven

// Также лямбда-функции обладают возможностью замыкать переменные
// Это позволяет сохранить используемый контекст для дальнейшего исполнения
val capture: Int = 5
val functionalCaputre: => Int = { capture }

...

functionalCapture() // == 5
// В таком случае они называются функциональными замыканиями
```

Анонимные функции

Определение

```
// Анонимные функции (или лямбда-функции) --- концепция, пришедшая из языков функциональной парадигмы
// Она позволяет по ходу программы создавать функции, которые не имеют уникального индентификатора
val filerEvenLambda: Int => Bool = { x: Int => x % 2 }  // == filterEven

// Часто они используются как предикаты или операторы при работе с коллекциями
val collection: Seq[Int] = ...
collection.filter(x => x % 2)        // Оставить в коллекции только четные числа
collection.sort(x, y => x > y)       // Остортировать коллекцию по убыванию
collection.foreach(x => println(x))  // Вывести все элементы коллекции
collection.map(x => x * x)           // Создать новую коллекцию, которая будет содержать квадраты чисел
...
```

Анонимные функции

Определение

```
// Анонимные функции (или лямбда-функции) --- концепция, пришедшая из языков функциональной парадигмы
// Она позволяет по ходу программы создавать функции, которые не имеют уникального индентификатора
val filerEvenLambda: Int => Bool = { x: Int => x % 2 }  // == filterEven

// Часто они используются как предикаты или операторы при работе с коллекциями
val collection: Seq[Int] = ...
collection.filter(x => x % 2)        // Оставить в коллекции только четные числа
collection.sort(x, y => x > y)       // Остортировать коллекцию по убыванию
collection.foreach(x => println(x))  // Вывести все элементы коллекции
collection.map(x => x * x)           // Создать новую коллекцию, которая будет содержать квадраты чисел
...
// Однако в Scala их использование куда шире на уровне языка
```

Анонимные функции

В Scala

1. Цикл `for`

```
for i in someSeq:
  println(i)
```

Данный цикл во время компиляции будет превращен в:

```
someSeq.foreach { i ⇒ println(i) }
```

Анонимные функции

В Scala

1. Цикл `for`
2. η-расширение

```
def increment(x: Int): Int = x + 1
val list = List(1, 2, 3)

list.map(increment)
```

Метод `increment` превращается в лямбду `{ x ⇒ increment(x) }`

Анонимные функции

В Scala

1. Цикл `for`
2. η-расширение
3. Параметры по имени (By-name parameters)

```
def log(message: ⇒ String): Unit = {
  if (debug) println(message)
}

log("User " + id + " logged in")
```

`"User " + id + " logged in"` превращается в лямбду `{ () ⇒ "User " + id + … }`
Преобразование в строку и конкатенация будут отложены до вызова функции `message`

Анонимные функции

В Scala

1. Цикл `for`
2. η-расширение
3. Параметры по имени (By-name parameters)
4. Частично примененные функции (Partially Applied Functions)

```
def sum(a: Int, b: Int): Int = a + b
val addFive = sum(5, _: Int)
```

`sum(5, _: Int)` превращается в лямбду `{ x ⇒ sum(5, x) }`

Анонимные функции

В Scala

1. Цикл `for`
2. η-расширение
3. Параметры по имени (By-name parameters)
4. Частично примененные функции (Partially Applied Functions)
5. Частично определенные функции (Partial Functions)
6. SAM-типы (Single Abstract Method)
7. Контекстные функции (Context Functions)

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
val lambda = { x ⇒ println(x) }
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
class Lambda$$1 extends (Int ⇒ Unit) {
  override def apply(x: Int) = {
    println(x)
  }---

}

val lambda = { x ⇒ println(x) }
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
class Lambda$$1 extends (Int ⇒ Unit) {
  override def apply(x: Int) = {
    println(x)
  }
}

val lambda = new Lambda$$1()
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
val capture = "arg = "
val lambda = { x ⇒ println(capture + x) }
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
class Lambda$$1(prefix: Str) extends (Int ⇒ Unit) {
  override def apply(x: Int) = {
    println(prefix + x)
  }
}

val capture = "arg = "
val lambda = new Lambda$$1(capture)
lambda(42)
```

Анонимные функции

Статистика

Название программы	Количество классов	Количество анонимных функций	Процент
Huawei VM	17247	7734	44.84%
Lisp	46	3	6.5%

Постановка задачи

Проблема

Каждая из анонимных функций в условиях *JVM* (*Java Virtual Machine*) — это полноценный объект, который требует как аллокации, так и сборки мусора. Из-за чего возникают издержки по памяти и времени.

Постановка задачи

Проблема

Каждая из анонимных функций в условиях *JVM* (*Java Virtual Machine*) — это полноценный объект, который требует как аллокации, так и сборки мусора. Из-за чего возникают издержки по памяти и времени.

Безусловно, существует набор оптимизаций, которые могут помочь в избавлении от создания объектов в куче.

Оптимизации

Анализ утеканий

Цель оптимизации — полностью или по полям переместить объект с кучи на стек. Это возможно сделать при соблюдении главного условия: объект не должен *утекать*, т.е. переходить в более широкую (относительно места создания) область видимости.

Оптимизации

Анализ утеканий

Цель оптимизации — полностью или по полям переместить объект с кучи на стек. Это возможно сделать при соблюдении главного условия: объект не должен *утекать*, т.е. переходить в более широкую (относительно места создания) область видимости.

Утеканиями будем считать:

Оптимизации

Анализ утеканий

Цель оптимизации — полностью или по полям переместить объект с кучи на стек. Это возможно сделать при соблюдении главного условия: объект не должен *утекать*, т.е. переходить в более широкую (относительно места создания) область видимости.

Утеканиями будем считать:

1. Присваивание в статическое поле, поле объекта или в массив

Оптимизации

Анализ утеканий

Цель оптимизации — полностью или по полям переместить объект с кучи на стек. Это возможно сделать при соблюдении главного условия: объект не должен *утекать*, т.е. переходить в более широкую (относительно места создания) область видимости.

Утеканиями будем считать:

1. Присваивание в статическое поле, поле объекта или в массив
2. Возврат из функции, если объект был создан внутри тела функции

Оптимизации

Анализ утеканий

Цель оптимизации — полностью или по полям переместить объект с кучи на стек. Это возможно сделать при соблюдении главного условия: объект не должен *утекать*, т.е. переходить в более широкую (относительно места создания) область видимости.

Утеканиями будем считать:

1. Присваивание в статическое поле, поле объекта или в массив
2. Возврат из функции, если объект был создан внутри тела функции
3. Передача в функцию, которую невозможно проанализировать